

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

MID SEMESTER EXAMINATION
DURATION: 2 HOURS

SUMMER SEMESTER, 2024–2025
FULL MARKS: 120

SWE 4201: Object Oriented Concepts I

Programmable calculators are not allowed. Do not write anything on the question paper.

Answer **all 4 (four)** questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

1. a) Explain the difference between an instance variable and a local variable, covering their scope and lifetime. Then write a short Java code snippet that clearly shows both in the same class.

6
(CO1)
(PO1)

Solution:

An **instance variable** belongs to an object and is declared inside a class but outside any method. Its scope is throughout the class, and it exists as long as the object exists. A **local variable** is declared inside a method, constructor, or block. Its scope is limited to that block, and it exists only while the block is executing.

```
1 class Demo {  
2  
3     int x = 10; // instance variable  
4  
5     void show() {  
6         int y = 20; // local variable  
7         System.out.println(x + y);  
8     }  
9 }
```

- b) Write a Java class called Rectangle that has length and width as private instance variables. Include a parameterized constructor, proper getters and setters, a method area() that returns the area. Also write a brief main() method that creates one Rectangle object and prints its area.

8
(CO3)
(PO3)

Solution:

```
1 class Rectangle {  
2  
3     private double length;  
4     private double width;  
5  
6     public Rectangle(double length, double width) {  
7         this.length = length;  
8         this.width = width;  
9     }  
10  
11     public double getLength() {  
12         return length;  
13     }  
14  
15     public void setLength(double length) {  
16         this.length = length;  
17     }  
18  
19     public double area() {  
20         return length * width;  
21     }  
22 }  
23  
24 public class Main {  
25     public static void main(String[] args) {  
26         Rectangle rect = new Rectangle(5, 10);  
27         rect.setLength(7);  
28         System.out.println("Area: " + rect.area());  
29     }  
30 }
```

```

17     }
18
19     public double getWidth() {
20         return width;
21     }
22
23     public void setWidth(double width) {
24         this.width = width;
25     }
26
27     public double area() {
28         return length * width;
29     }
30 }
31
32 public class Main {
33
34     public static void main(String[] args) {
35
36         Rectangle r = new Rectangle(5, 4);
37
38         System.out.println(r.area());
39     }
40 }

```

- c) Explain what a default constructor is and what happens if you define a parameterized constructor in a class without also explicitly defining a default one. Support your answer with a small Java example that demonstrates the compile-time consequence.

8
(CO1)
(PO1)

Solution:

A **default constructor** is a constructor that takes no parameters.

If a class does not define any constructor, Java automatically provides a default constructor. However, if a programmer defines a parameterized constructor, Java no longer generates a default constructor automatically. Attempting to create an object using a no-argument constructor will then result in a compile-time error.

```

1 class Student {
2
3     Student(String name) {
4
5     }
6 }
7
8 public class Main {
9
10     public static void main(String[] args) {
11
12         Student s = new Student(); // Compile-time error
13     }
14 }

```

The program fails to compile because the class defines only a parameterized constructor and no default constructor exists.

- d) Consider the following statements about OOP. For each one, state whether it is true or false and write one sentence justifying your answer.
- i. A subclass inherits the private fields of its superclass and can access them directly.
 - ii. Method overloading and method overriding are the same concept — both allow a class to have multiple methods with the same name.
 - iii. Making all fields of a class public improves encapsulation because it simplifies access to data.
 - iv. A constructor must always have the same name as the class it belongs to.

4 × 2
(CO1)
(PO1)

Solution:

i. **False.**

A subclass inherits the private fields of its superclass, but it cannot access them directly because private members are accessible only within the superclass itself.

ii. **False.**

Method overloading and method overriding are different concepts; overloading occurs within the same class using different parameter lists, whereas overriding redefines an inherited method in a subclass.

iii. **False.**

Public fields violate encapsulation because they allow unrestricted direct access to an object's data. Encapsulation is better maintained by keeping fields private and providing controlled access through methods.

iv. **False.**

A constructor must have the same name as its class; otherwise, it is treated as a regular method.

2. You are building a simple fantasy combat game called `Battle Arena`. Every fighter has a name, health points (HP), and attack power. A fighter can attack another fighter, reducing the target's HP by its attack power. HP can never become negative. A fighter is considered alive if its HP is greater than zero.

A `Hero` is a special type of `Fighter`. After every attack, a hero earns 10 gold coins. In addition, every third attack made by a hero is automatically a `Power Strike`, which deals twice the hero's normal attack power instead of a normal attack.

- a) Write the `Fighter` class based on the requirements above. Your implementation should include appropriate instance variables, constructor(s), methods, getters and setters needed to support the described behavior.

10
(CO3)
(PO3)

Solution:

```
1 class Fighter {
2
3     private String name;
4     private int hp;
5     private int attackPower;
6
7     public Fighter(String name, int hp, int attackPower) {
8         this.name = name;
9         setHp(hp);
10    }
```

```

10     this.attackPower = attackPower;
11 }
12
13 public void attack(Fighter target) {
14     target.setHp(target.getHp() - attackPower);
15 }
16
17 public boolean isAlive() {
18     return hp > 0;
19 }
20
21 public String getName() {
22     return name;
23 }
24
25 public void setName(String name) {
26     this.name = name;
27 }
28
29 public int getHp() {
30     return hp;
31 }
32
33 public void setHp(int hp) {
34     if(hp < 0) {
35         this.hp = 0;
36     }
37     else {
38         this.hp = hp;
39     }
40 }
41
42 public int getAttackPower() {
43     return attackPower;
44 }
45
46 public void setAttackPower(int attackPower) {
47     this.attackPower = attackPower;
48 }
49 }

```

- b) Using `Fighter` as the base class, write the `Hero` class. Include both a parameterized constructor and a non-parameterized constructor that creates a hero named "Warrior" with 100 health points and 15 attack power. The non-parameterized constructor must delegate to the parameterized constructor using `this()`. 12
(CO3)
(PO3)

Solution:

```

1 class Hero extends Fighter {
2
3     private int goldCoins;
4     private int attackCount;
5

```

```

6     public Hero() {
7         this("Warrior", 100, 15);
8     }
9
10    public Hero(String name, int hp, int attackPower) {
11        super(name, hp, attackPower);
12        goldCoins = 0;
13        attackCount = 0;
14    }
15
16    @Override
17    public void attack(Fighter target) {
18
19        attackCount++;
20
21        int damage = getAttackPower();
22
23        if(attackCount % 3 == 0) {
24            damage *= 2;
25        }
26
27        int newHp = target.getHp() - damage;
28
29        if(newHp < 0) {
30            newHp = 0;
31        }
32
33        target.setHp(newHp);
34
35        goldCoins += 10;
36    }
37
38    public int getGoldCoins() {
39        return goldCoins;
40    }
41 }

```

- c) Suppose the game is later expanded to include a new class called Mage. Like a Hero, a Mage is a type of Fighter. Unlike a hero, every fourth attack is a magic spell that deals three times the mage's normal attack power. Instead of collecting gold coins, a mage collects magic crystals after each attack. 8
(CO2)
(PO2)

Identify which behaviors belong in `Fighter` and which belong in subclasses, including how the special attack rules for `Hero` and `Mage` should be implemented.

Solution:

To minimize code duplication, all common properties and behaviors should remain in the `Fighter` superclass. These include the fighter's name, HP, attack power, the ability to determine whether a fighter is alive, and the ability to attack another fighter.

The subclasses should contain only the behavior that is unique to them.

- `Hero` should store the number of gold coins earned and implement the special rule that every third attack is a `Power Strike` dealing double damage.

- Mage should store the number of magic crystals earned and implement the special rule that every fourth attack is a spell dealing triple damage.

A good design is to let each subclass override the attack behavior while reusing the common state and functionality inherited from `Fighter`. This avoids duplicating fields such as name, HP, attack power, and alive-status logic in every subclass.

3. a) Explain the purpose of abstract classes and describe a situation where they are useful.

8
(CO1)
(PO1)

Solution:

Abstract classes are useful even though they cannot be instantiated directly. Their purpose is to provide a common blueprint for related classes while preventing the creation of incomplete objects.

Benefits of abstract classes include:

- They allow common fields and methods to be shared among subclasses.
- They can define abstract methods that force subclasses to provide their own implementations.
- They prevent objects from being created from incomplete or overly general classes while still allowing common functionality to be shared by subclasses.

For example, in a drawing application, a class `Shape` may contain common properties such as color and abstract methods such as `draw()`. Concrete subclasses such as `Circle` and `Rectangle` provide their own implementations of `draw()`. Since a generic shape should not be created directly, `Shape` is an appropriate abstract class.

- b) Consider the `main()` method shown in Code Snippet 1.

14
(CO3)
(PO3)

```

1 public class Main {
2     public static void main(String[] args) {
3         Movie m1 = new Movie("Inception", "Christopher Nolan", 2010);
4
5         Movie m2 = new Movie("Interstellar");
6
7         m1.setRating(9);
8         m2.setRating(7);
9
10        System.out.println(Movie.getMovieCount());
11        System.out.println(m1.getSummary());
12        System.out.println(m2.getSummary());
13
14        System.out.println("Inception newer? " + m1.isNewerThan(m2));
15    }
16 }
```

Code Snippet 1: `main()` for Question 3.b

The requirements of the `Movie` class are -

- `getSummary()` should return a descriptive string about the movie.
- `isNewerThan(Movie other)` should compare release years and return a boolean result.

- Since `m2` is created without a release year, choose an appropriate default value and clearly state your assumption.
- Keep track of the total number of `Movie` objects created and provide access through `getMovieCount()`.
- Ratings must be between 0 and 10. Implement a helper method for validation and use it from the setter. When an invalid rating is provided, store the nearest valid value.

Using the code above and the stated requirements, write the `Movie` class. Infer any necessary fields, constructors, and methods from how the class is used.

Solution:

Assumption: If only the title is provided, the director is set to "Unknown" and the release year is set to 0.

```

1 class Movie {
2
3     private String title;
4     private String director;
5     private int year;
6     private int rating;
7     private static int movieCount = 0;
8
9     public Movie(String title, String director, int year) {
10
11         this.title = title;
12         this.director = director;
13         this.year = year;
14         this.rating = 0;
15         movieCount++;
16     }
17
18     public Movie(String title) {
19         this(title, "Unknown", 0);
20     }
21
22     private int validateRating(int rating) {
23
24         if(rating < 0) {
25             return 0;
26         }
27
28         if(rating > 10) {
29             return 10;
30         }
31
32         return rating;
33     }
34
35     public void setRating(int rating) {
36         this.rating = validateRating(rating);
37     }
38
39     public int getRating() {
40         return rating;

```

```

41     }
42
43     public String getSummary() {
44
45         return title + " directed by " + director + ", released in "
46             + year + ", rating " + rating;
47     }
48
49     public boolean isNewerThan(Movie other) {
50         return this.year > other.year;
51     }
52
53     public static int getMovieCount() {
54         return movieCount;
55     }

```

c) Answer the following questions about method overloading.

- i. Explain what method overloading is. State two requirements that must be satisfied for two methods to be considered overloaded.
- ii. A student claims that changing only the return type of a method is sufficient to create an overloaded method. Do you agree or disagree? Justify your answer.
- iii. Explain ambiguity in method overloading. Briefly describe a situation where an overloaded method call may become ambiguous.

3 +
3 + 2
(CO1)
(PO1)

Solution:

- a) Method overloading occurs when multiple methods in the same class have the same name but different parameter lists.

Two requirements are:

- The methods must have the same name.
- The methods must differ in the number, type, or order of parameters.

- b) I disagree. Changing only the return type does not create an overloaded method because Java determines which method to call using the method signature, which does not include the return type.

The following code produces a compile-time error:

```

1 int calculate() {
2     return 10;
3 }
4
5 double calculate() {
6     return 10.0;
7 }

```

- c) Ambiguity occurs when the compiler cannot determine which overloaded method should be called.

For example:

```

1 void show(int x, double y) { }
2
3 void show(double x, int y) { }
4
5 show(5, 5);

```

The call `show(5, 5)` can match both methods equally well, causing a compile-time ambiguity error.

4. a) “Encapsulation just means making variables private — it adds no real benefit, it only makes code harder to write.” Do you agree with this statement? Justify your answer with at least two reasons.

6
(CO2)
(PO2)

Solution:

I disagree.

- Encapsulation protects the internal state of an object from unauthorized or accidental modification. By making variables private, the class controls how its data can be accessed and updated.
- Encapsulation improves maintainability. If the internal implementation changes, other classes do not need to be modified as long as the public interface remains the same.
- Encapsulation allows validation of data through methods such as setters. For example, a bank account class can prevent a negative balance from being assigned.
- Encapsulation reduces coupling between classes and makes software easier to understand, test, and maintain.

Therefore, encapsulation provides significant benefits beyond simply making variables private.

- b) Consider the program shown in Code Snippet 2 and answer the questions that follow.

```

1 class Account {
2
3     public double fee() {
4         return 100;
5     }
6 }
7
8 class PremiumAccount extends Account {
9
10    @Override
11    public double fee() {
12        return 50;
13    }
14 }
15
16 public class Main {
17
18     public static void main(String[] args) {
19

```

4 + 4 +
4 + 2
(CO2)
(PO2)

```

20     Account[] accounts = {
21         new Account(),
22         new PremiumAccount(),
23         new PremiumAccount()
24     };
25
26     double total = 0;
27
28     for(Account a : accounts) {
29         total += a fee();
30     }
31
32     System.out.println(total);
33 }
34 }

```

Code Snippet 2: Program for Question 4.b

- i. Write the exact output of the program.
- ii. Explain how the correct `fee()` method is chosen for each object. Name the OOP concept involved.
- iii. Suppose `fee()` were declared `static` in both classes. Would the output change? Explain your answer.
- iv. What advantage does using an `Account[]` array provide compared to using an array of `PremiumAccount` objects?

Solution:

- i. The output is 200.0.

Explanation:

- The first object is an `Account`, so `fee()` returns 100.
- The second object is a `PremiumAccount`, so `fee()` returns 50.
- The third object is also a `PremiumAccount`, so `fee()` returns 50.

Therefore,

$$100 + 50 + 50 = 200$$

and the program prints 200.0.

- ii. The correct `fee()` method is selected using **runtime polymorphism** (dynamic method dispatch). Although the array stores references of type `Account`, the actual method executed depends on the real type of the object at runtime.
 - For an `Account` object, `Account.fee()` is called.
 - For a `PremiumAccount` object, `PremiumAccount.fee()` is called.
- iii. Yes, the output would change. If `fee()` were declared `static`, method overriding would no longer occur because static methods cannot be overridden; they are hidden instead. Static method calls are resolved at **compile time** using the reference type. Since every element in the array has the reference type `Account`, each call to `a.fee()` would invoke `Account::fee()`, regardless of the actual object.

Thus,

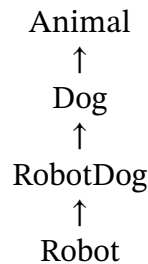
$$100 + 100 + 100 = 300$$

and the output would become `300.0`.

- iv. Using an `Account[]` allows the array to store both `Account` objects and objects of any subclass such as `PremiumAccount`. This provides flexibility and supports polymorphism. A `PremiumAccount[]` array could store only `PremiumAccount` objects and could not hold regular `Account` objects.

c) A student proposes the following hierarchy:

10
(CO2)
(PO2)



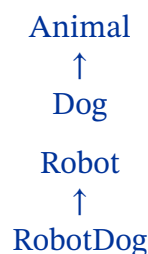
Critically evaluate this inheritance hierarchy. If you believe it can be improved, explain how you would redesign it and justify your changes.

Solution:

The hierarchy has several design problems.

- A `Dog` is an `Animal`, so that relationship is valid.
- A `RobotDog` may resemble a dog, but it is not necessarily a biological animal. Therefore inheriting it from `Dog` may violate the IS-A relationship.
- The biggest problem is that `Robot` inherits from `RobotDog`. A general robot is not a type of robot dog. Therefore the inheritance relationship is incorrect.
- The hierarchy mixes biological and mechanical concepts, resulting in poor design and weak abstraction.

A better design is:



This design preserves proper IS-A relationships, improves maintainability, and follows sound object-oriented design principles.

MARKS SUMMARY TABLE

Question	Marks	Keywords
1.a	6	
1.b	8	
1.c	8	
1.d	8	
2.a	10	
2.b	12	
2.c	8	
3.a	8	
3.b	14	
3.c	8	
4.a	6	
4.b	14	
4.c	10	
Total	120	